

LAPORAN PRAKTIKKUM TEKNOLOGI IoT

Antarmuka Sensor dan Sistem Kendali Dasar



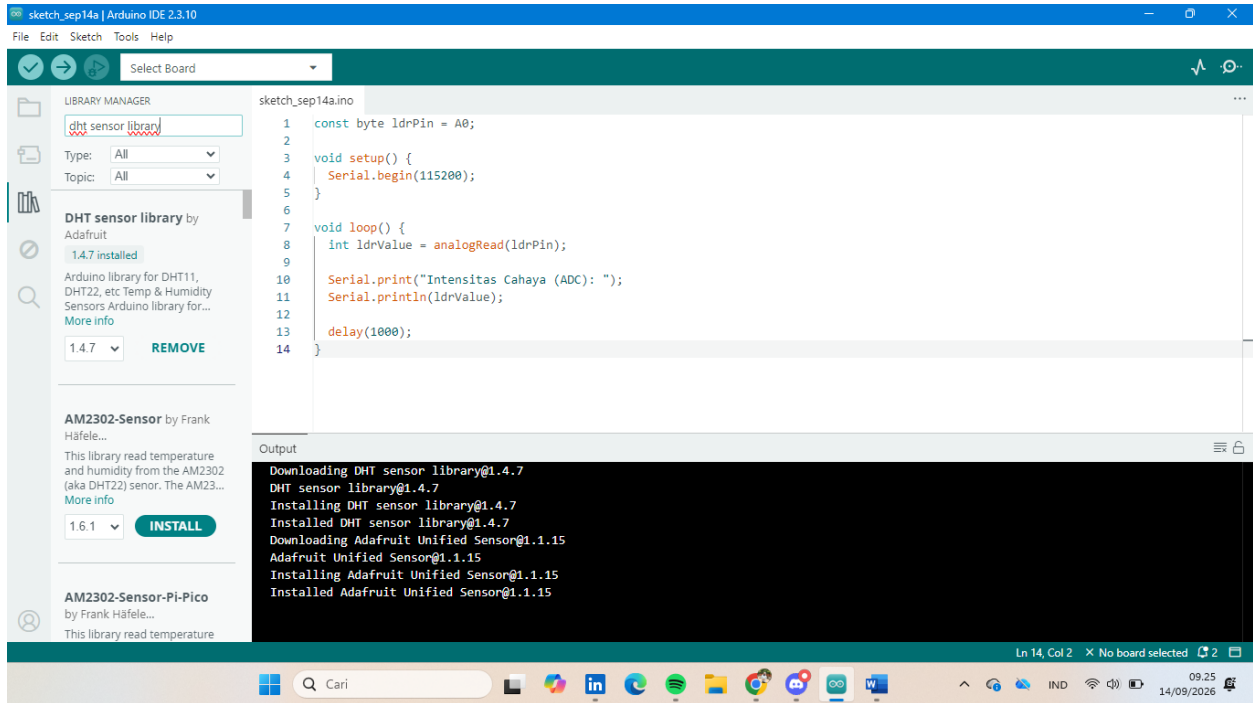
Nama: Novita Syifa Azhara
NPM: 25782078
Kelas: 3C

Dosen: Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG**

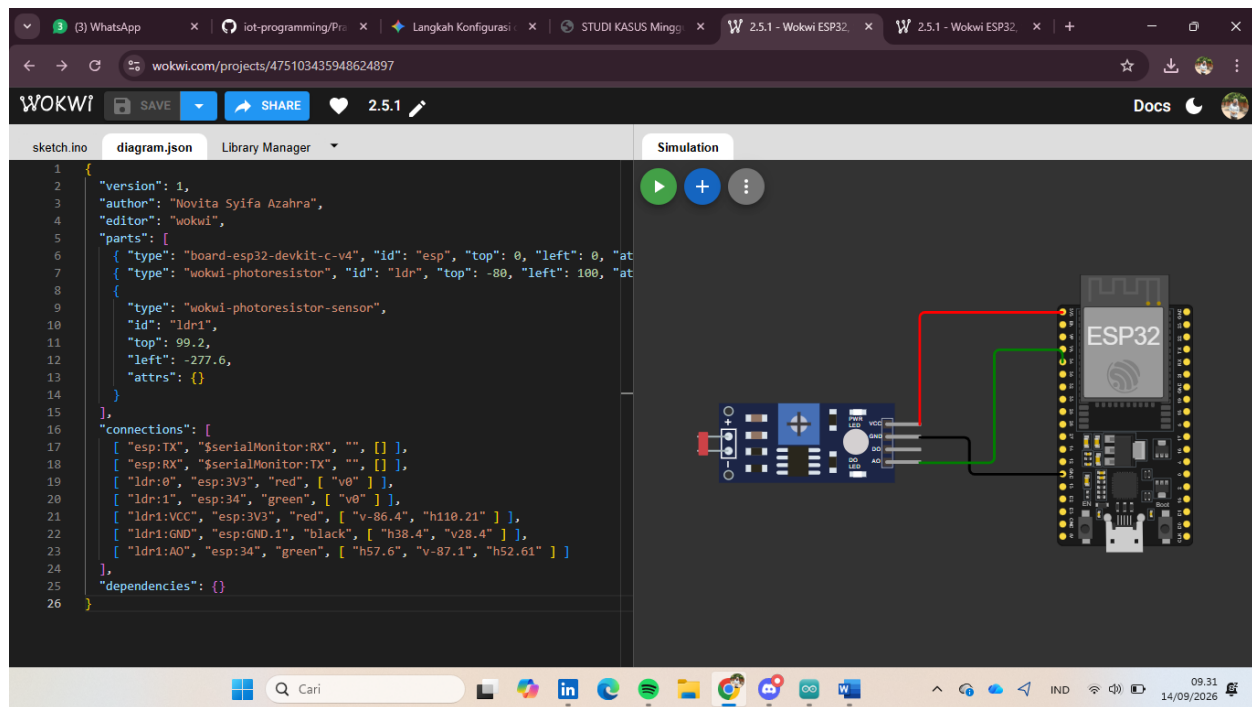
2.4.2 Persiapan Perangkat Lunak (Instalasi Library DHT)

Membaca sinyal digital single-bus dari sensor DHT sangat rumit jika dilakukan manual karena membutuhkan presisi waktu dalam hitungan microsecond. Oleh karena itu, kita wajib menginstal perangkat lunak tambahan berupa Library.



2.5.1 Praktikum 1: Pembacaan Intensitas Cahaya (LDR) via ADC

Sensor LDR akan menurunkan hambatannya (resistansi) saat terpapar cahaya terang, dan meningkatkan resistansinya saat gelap. Untuk menerjemahkan perubahan resistansi ini menjadi variasi tegangan yang bisa diukur oleh pin A0, kita menggunakan sirkuit pembagi tegangan (Voltage Divider).

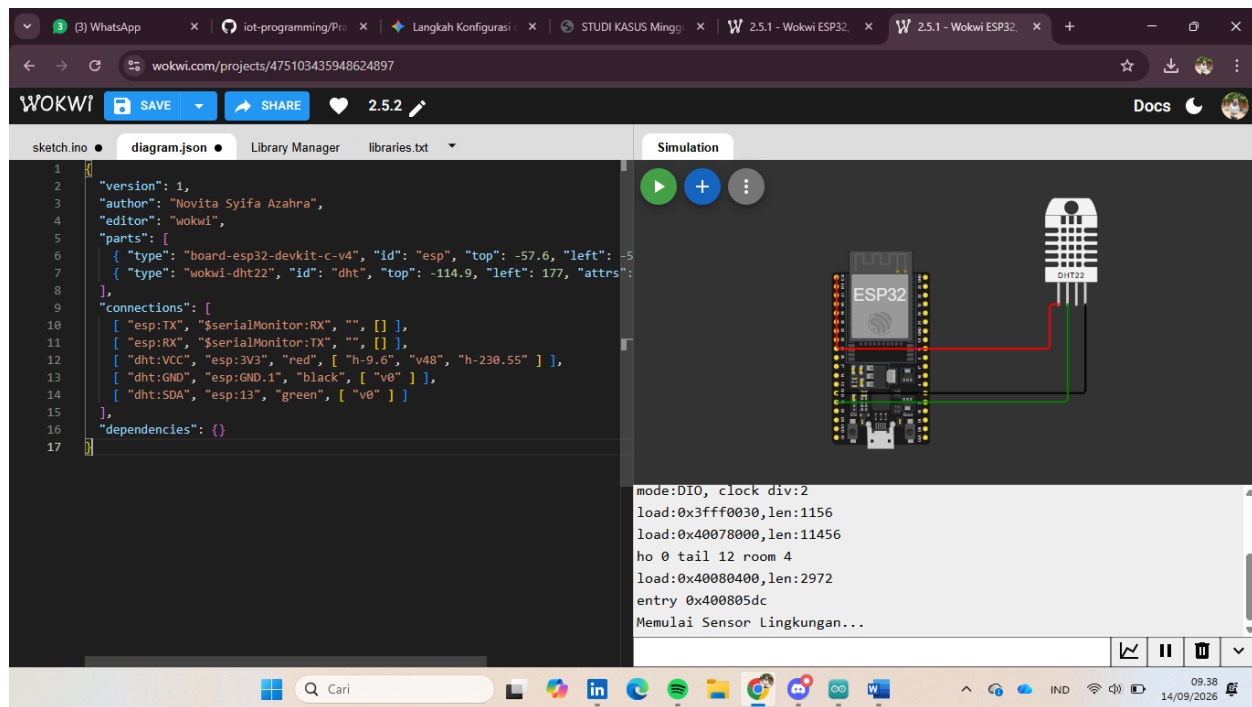


<https://wokwi.com/projects/475109275957177345>

- (Logika Rangkaian: Saat cahaya terang, resistansi LDR kecil sehingga arus 3.3V membanjiri A0, menghasilkan nilai ADC besar mendekati 1023. Saat gelap, resistansi LDR sangat besar menahan arus, sehingga pin A0 'tertarik' oleh resistor 10k ke arah GND, menghasilkan nilai ADC kecil mendekati 0).
- Penjelasan Singkat Kode: `const byte ldrPin = A0;` Mendeklarasikan pin ADC. Tipe byte digunakan (menggantikan int) untuk menghemat memori (RAM), karena ID pin tidak akan melebihi angka 255. `analogRead(ldrPin);` Fungsi bawaan mikrokontroler untuk membaca level tegangan analog murni di pin A0 dan mengonversinya menjadi data integer 10-bit (berkisar antara 0 hingga 1023).

2.5.2 Praktikum 2: Membaca Data Sensor Suhu & Kelembapan (DHT)

Sensor DHT menggunakan protokol komunikasi digital single-wire. Artinya, seluruh data (suhu dan kelembapan) ditransmisikan melalui satu kabel data saja. Protokol ini sangat bergantung pada waktu (timing). Mikrokontroler harus mengirimkan perintah "minta data" dengan durasi HIGH dan LOW yang sangat presisi. Setelah itu, sensor akan mengirimkan kembali serangkaian pulsa digital yang panjangnya spesifik menunjukkan nilai suhu dan kelembapan.



<https://wokwi.com/projects/475109469640774657>

Penjelasan Singkat Kode:

- `include <DHT.h>`: Mengimpor fungsi-fungsi dari library Adafruit DHT yang baru saja Anda instal.
- `define DHTTYPE DHT11`: Mendefinisikan konstanta preprosesor untuk tipe sensor. (Ubah menjadi DHT22 jika sensor Anda berwarna putih).
- `DHT dht(dhtPin, DHTTYPE);`: Membentuk "objek" pengendali bernama dht yang mengingat pin data dan jenis sensor yang digunakan.
- `float temp = dht.readTemperature();`: Perintah ini bertugas membaca suhu dan menyimpannya dalam variabel float (karena mengandung angka desimal).
- `isnan(temp)`: Is Not a Number. Fungsi matematika yang bertindak sebagai sistem perlindungan (error handling). Jika kabel sensor putus, library akan menghasilkan data rusak (Not-A-Number). Blok if ini bertugas mendeteksi kerusakan tersebut dan membatalkan sisa kode di bawahnya dengan perintah `return;`.

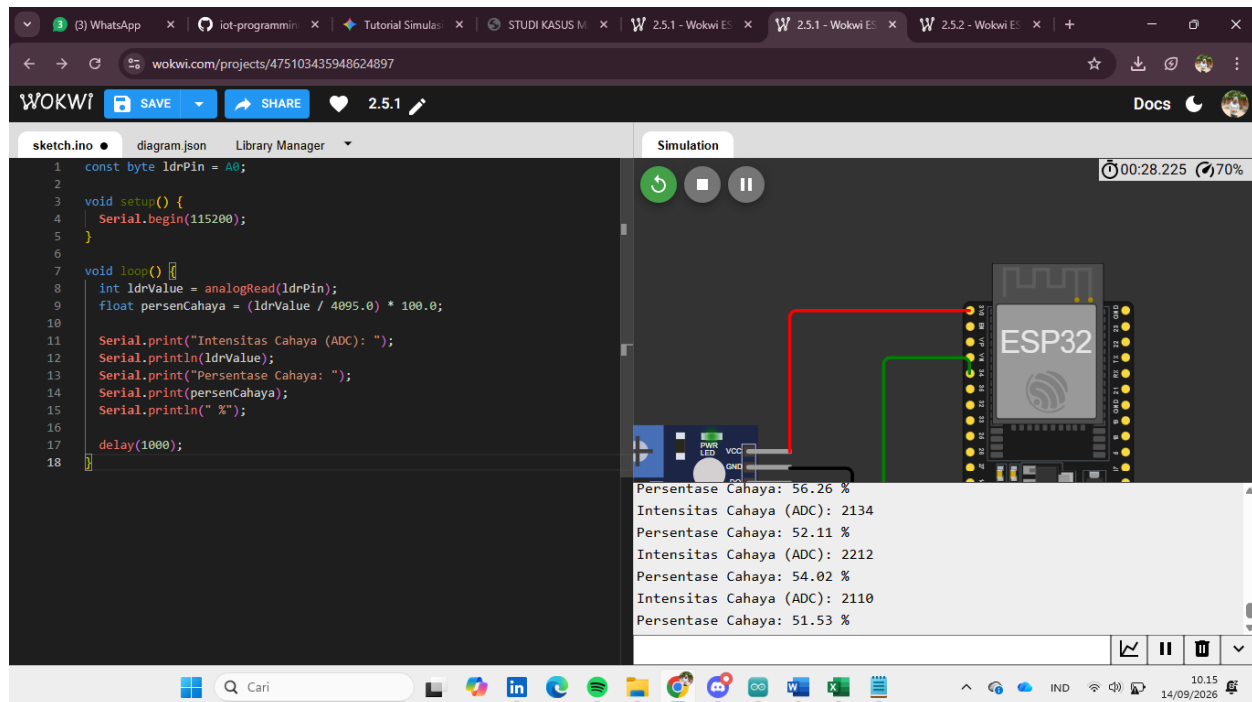
2.6 Latihan

Kerjakan latihan berikut untuk menguji retensi konsep Anda secara mandiri.

Latihan 1 (Pemahaman Sistem ADC): Mengapa batas maksimal nilai intensitas cahaya yang terbaca pada Serial Monitor di Praktikum 1 adalah sekitar angka 1023 (bukan 100 atau 1000)? Kaitkan jawaban Anda dengan penjelasan "Resolusi ADC 10-bit". **Batas maksimal nilai intensitas cahaya adalah 1023**

karena resolusi ADC pada ESP8266 adalah **10-bit**. Berdasarkan perhitungan digital ($2^{10} = 1024$), rentang nilai konversi yang dihasilkan dimulai dari angka 0 hingga 1023 ($1024 - 1$). Oleh karena itu, angka maksimalnya adalah 1023, bukan 100 atau 1000.

Latihan 2 (Implementasi Algoritma): Nilai ADC 0-1023 tidak lazim dibaca oleh pengguna akhir (manusia). Modifikasi kode pada Praktikum 1 agar program menghitung dan mencetak persentase intensitas cahaya (rentang 0% hingga 100%). Tuliskan rumus matematika / perintah pembagian yang Anda sematkan di kode tersebut!



<https://wokwi.com/projects/475109701121399809>

Untuk mengonversi rentang nilai ADC (0–1023) menjadi persentase (0% hingga 100%), rumus matematika yang digunakan adalah membagi nilai pembacaan dengan angka maksimum lalu dikalikan 100.

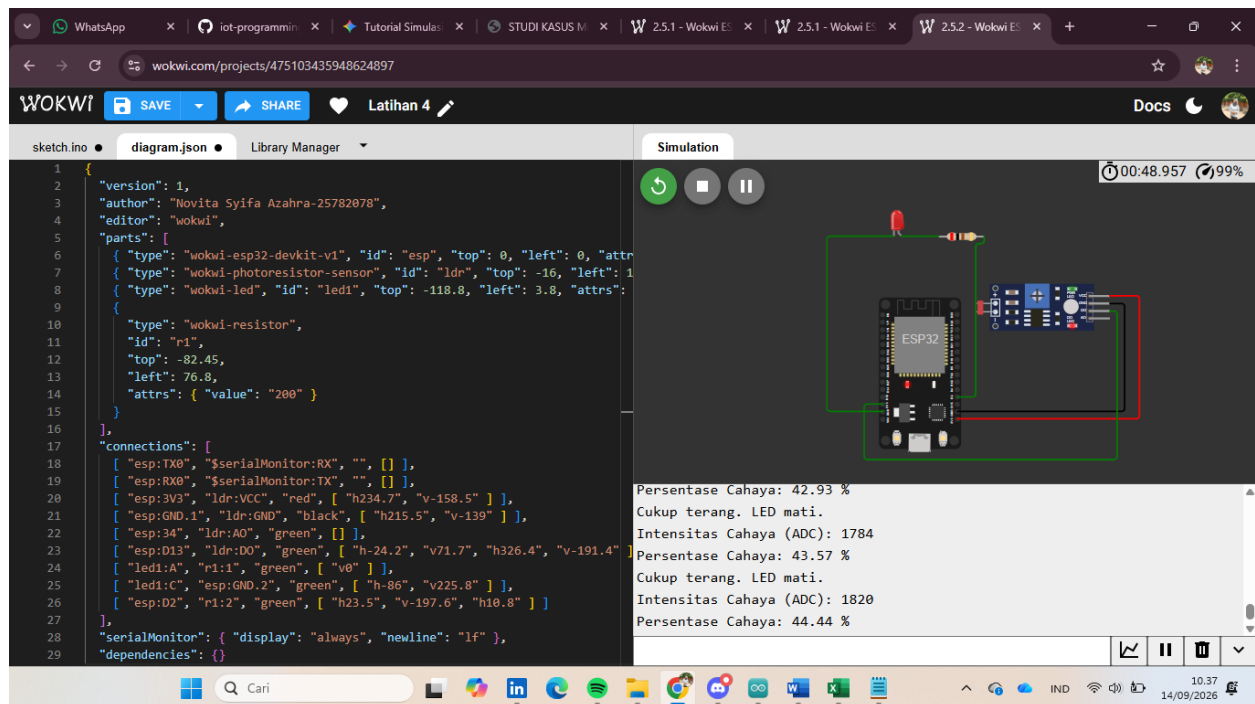
- **Rumus Matematika:** $\text{Persentase} = \left(\frac{\text{ldrValue}}{1023.0} \right) \times 100$
- **Perintah Pemrograman / Pembagian:**

```
int lightPercent = (ldrValue / 1023.0) * 100;
// Atau menggunakan fungsi map bawaan Arduino:
// int lightPercent = map(ldrValue, 0, 1023, 0, 100);
```

Latihan 3 (Analisis Rangkaian): Di subbab 2.3.4 disebutkan sifat modul Relay yang terisolasi Optocoupler. Banyak Relay di pasaran yang bekerja secara Active-Low. Jika Relay Anda bertipe Active-Low, apa yang terjadi pada aktuator (misal: lampu ruangan) yang terhubung ke Relay jika program Anda mengeksekusi `digitalWrite(relayPin, HIGH)`? Jika modul relay bertipe **Active-**

Low, eksekusi perintah `digitalWrite(relayPin, HIGH)` akan **mematikan relay** (memutus sirkuit), sehingga aktuator (seperti lampu ruangan) yang terhubung ke relay akan **padam/mati**. Relay tipe **Active-Low** hanya akan aktif menyalakan sirkuit tegangan tinggi jika menerima sinyal **LOW**.

Latihan 4 (Pengembangan Logika): Gabungkan wawasan dari Praktikum 1 Modul 2 ini dengan logika kendali LED dari Praktikum Modul 1. Gunakan blok kondisional `if` yang berfungsi menyalakan sebuah LED (D1) **HANYA JIKA** sensor LDR mendeteksi lingkungan yang gelap gulita (misalnya: ambang batas ADC berada di bawah angka 200).



<https://wokwi.com/projects/475103435948624897>

Penerapan blok kondisional `if` untuk menyalakan LED pada pin D1 (GPIO 5) hanya ketika sensor LDR mendeteksi gelap gulita di bawah angka 200 dapat dituliskan melalui kode berikut:

```
const byte ldrPin = A0;
```

```
const byte ledPin = 5; // D1 pada NodeMCU
```

```
void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Amankan kondisi awal LED mati
}
```

```
void loop() {
  int ldrValue = analogRead(ldrPin);
  // Nyalakan LED HANYA JIKA lingkungan gelap gulita di bawah angka 200
  if (ldrValue < 200) {
    digitalWrite(ledPin, HIGH);
  } else {
```

```
digitalWrite(ledPin, LOW);  
}  
  
delay(1000);  
}
```

2.7 Tugas: Sistem Otomasi Kontrol Lokal Terpadu

Pertahankan sirkuit sensor LDR (di A0) dan DHT (di D7).

Hubungkan pin VCC / +5V dari Modul Relay ke pin VIN atau VU NodeMCU (mengambil daya murni 5V dari USB). Jangan menyambungkan modul Relay 5V ke pin 3V3!

Hubungkan pin GND Relay ke jalur GND bersama NodeMCU.

Hubungkan pin IN / Signal Relay ke pin D6 (GPIO 12).

(Opsional sebagai indikator visual tambahan): Pasang sebuah LED beresistor 220 Ohm pada pin D1 (GPIO 5).

Instruksi Pemrograman (Aturan/Rule Engine):

Konfigurasi pin Relay (D6) dan LED (D1) sebagai OUTPUT. (Kirimkan perintah mati di awal fungsi setup() untuk keamanan).

Buatlah struktur logika perbandingan (if ... else) menggunakan operator logika OR (||).

Aktifkan Relay dan nyalakan LED indikator JIKA kondisi lingkungan buruk berikut terpenuhi:

Suhu udara terbaca di atas 34 Celcius (Terlalu panas).

ATAU Nilai ADC LDR terbaca di bawah angka 300 (Terlalu gelap).

Jika suhu normal dan ruangan cukup cahaya, pastikan Relay dan LED kembali mati.

Cetak status sistem ke Serial Monitor secara rutin (Contoh: "Kondisi Aman" / "Peringatan: Aktuator Aktif!").

Kriteria Keberhasilan:

Sensor LDR dan DHT berhasil dibaca bersamaan dalam satu loop tanpa interupsi atau kegagalan pembacaan (NaN).

Logika kondisi OR beroperasi dengan presisi (cukup salah satu syarat buruk terpenuhi, aktuator menyala).

Relay bekerja selaras dan logis sesuai dengan sifat modul kelistrikan tersebut (pahami apakah modul Anda bersifat Active-Low atau Active-High).

<https://wokwi.com/projects/475110009964900353>

Wokwi project simulation interface showing a JSON configuration for a circuit and its visual representation.

JSON Configuration (diagram.json):

```
1 {
2   "version": 1,
3   "author": "Novita Syifa Azahra",
4   "editor": "wokwi",
5   "parts": [
6     { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 4.84, "attrs": {} },
7     { "type": "wokwi-photoresistor-sensor",
8       "id": "ldr",
9       "top": 166.4,
10      "left": -258.4,
11      "attrs": {} },
12     { "type": "wokwi-dht22", "id": "dht1", "top": -120, "left": -40, "attrs": {} },
13     { "type": "wokwi-relay-module", "id": "relay1", "top": 153.8, "left": 172.8, "attrs": {} },
14     { "type": "wokwi-led", "id": "led1", "top": -109.2, "left": 167, "attrs": { "color": "blue" } },
15     { "type": "wokwi-resistor",
16       "id": "r1",
17       "top": -44.05,
18       "left": 96,
19       "attrs": { "value": "220" } },
20   ],
21   "connections": [
22     [ "esp:TX", "$SerialMonitor:RX", "", [] ],
23     [ "esp:RX", "$SerialMonitor:TX", "", [] ],
24     [ "ldr:VCC", "esp:3V3", "red", [ "v0" ] ],
25     [ "ldr:GND", "esp:GND.1", "black", [ "v0" ] ],
26     [ "ldr:AO", "esp:34", "green", [ "h38.4", "v-144.7" ] ],
27     [ "dht1:VCC", "esp:3V3", "red", [ "v0" ] ],
28     [ "dht1:GND", "esp:GND.1", "black", [ "h-13.4", "v158.7" ] ],
29     [ "relay1:VCC", "esp:VIN", "red", [ "v0" ] ],
30   ]
31 }
```

Simulation View: A visual representation of the circuit components and their connections. The components include an ESP32 board, a photoresistor sensor (ldr), a DHT22 temperature and humidity sensor (dht1), a relay module (relay1), an LED (led1), and a 220Ω resistor (r1). The connections are color-coded: red for VCC, black for GND, and green for signal lines.

System Information: The interface shows the Wokwi logo, a 'SAVE' button, a 'SHARE' button, and a version number '2.7'. The browser address bar displays 'wokwi.com/projects/47511000964900353'. The system tray at the bottom indicates the time as 11:11 on 14/09/2026.